COE 619 Progress Report I

Energy-Aware Predictive VM Consolidation for Reducing Energy Consumption and SLA Violations in Cloud Data Centers

Student: Zahid Alabadllah (**ID:** 202045160)

Department of Computer Engineering, King Fahd University of Petroleum & Minerals

Term 261 – Week 10

Primary Advisor: Dr. Mohamed Faten Zhani

1 Summary of Progress

This report documents the completion of the first phase of the COE 619 project. In this initial stage, the foundational software simulation environment has been deployed, verified, and benchmarked using two standard reactive baseline consolidation policies. The predictive machine learning heuristic outlined in the project proposal will be formulated and implemented in the subsequent project phase. The completed deliverables include:

- **Simulation Framework:** A modern Java/Maven simulation platform utilizing CloudSim Plus 8.0.0 [2] with JDK 25.
- **Baseline Consolidation Policies:** Fully operational implementations of Static Threshold (THR) and dynamic Interquartile Range (IQR) policies [1].
- **Evaluation Metrics:** Implementation and output of Data Center Energy Consumption, SLA Time per Active Host (SLATAH), Performance Degradation due to Migration (PDM), Overall SLA Violations (SLAV), Number of VM Migrations, and Energy–SLA Violations product (ESV).
- **Resource Accounting & Metric Formulations:** Resolution of host RAM/bandwidth reservation inconsistencies during migration evaluation and normalization of host CPU capacity utilization.

2 Problem and Objectives

Modern cloud data centers host diverse computational workloads requiring substantial electrical power. A lightly loaded server still draws a large share of its peak power [1]. Dynamic Virtual Machine (VM) consolidation utilizes live migration [4] to aggregate active workloads onto the minimum feasible subset of physical hosts, allowing underutilized physical servers to transition into low-power sleep states.

However, aggressively packing VMs introduces significant performance risks. Sudden workload surges can fully saturate available host CPU processing capacity, so VMs may receive less CPU capacity than they request (unmet demand), which can violate the Service Level Agreement (SLA) between provider and customer.

Standard reactive heuristics trigger VM migrations only *after* a physical host has already exceeded a predefined threshold. The primary objective of this project is to develop and evaluate an intelligent, lightweight *predictive* consolidation policy that anticipates CPU utilization spikes and migrates VMs proactively *before* overload conditions manifest. No claim is made at this stage that prediction will improve on the baselines. The objective of Phase I was to build, calibrate, and verify the reactive reference baselines.

3 Background and Related Work

Virtualization and Live Migration. Hypervisors (Virtual Machine Monitors) enable concurrent, isolated virtual machine execution over shared physical compute resources [7]. Live migration moves a running VM between physical hosts. The conventional *pre-copy* protocol copies memory pages iteratively across the network while the virtual machine continues computation, followed by a brief stop-and-copy pause phase [4]. Because pre-copy operations introduce memory-dirtying overhead and consume CPU cycles and interconnect bandwidth, VM application performance experiences transient degradation. Voorsluys et al. [5] measured this degradation for a web workload, and Beloglazov and Buyya [1] adopted an estimate of about 10% of the VM's CPU capacity during migration. In this project the 10% figure is a modeling assumption, not a measurement.

Reactive Consolidation Heuristics. Beloglazov and Buyya [1] decoupled dynamic VM consolidation into four distinct sub-problems: (i) host overload detection, (ii) host underload detection, (iii) VM selection for migration, and (iv) target host placement. For host overload detection, they analyzed static utilization thresholds (THR) against adaptive statistical heuristics, including Median Absolute Deviation (MAD) and Interquartile Range (IQR), as well as regression-based techniques (Local Regression and Robust Local Regression). Their benchmark suite utilized CloudSim [3] driven by real-world CoMon PlanetLab CPU traces [6]. This project establishes a direct baseline comparison against these classical heuristics prior to introducing forward-looking predictive models.

4 Implemented Baselines

The two implemented baselines share an identical physical architecture, differing strictly in their mathematical upper overload detection trigger. Both baselines execute the standard Minimum Migration Time (MMT) selection policy, which selects the active VM requiring the shortest estimated migration duration [1].

Static Threshold (THR).A physical host is flagged as overloaded whenever its aggregate instantaneous CPU utilization u exceeds a fixed threshold $T_u = 0.80$ (80% of physical host processing capacity).

Interquartile Range (IQR). Rather than relying on a static limit, the upper threshold dynamically adapts to the statistical dispersion of each host's historical CPU utilization:

$$T_u = 1 - s \cdot \mathrm{IQR}, \qquad \mathrm{IQR} = Q_3 - Q_1, \tag{1}$$

where Q_1 and Q_3 denote the 25th and 75th percentiles (first and third quartiles) of recent host CPU utilization samples, and s represents a safety parameter [1]. Hosts exhibiting higher demand variance receive a reduced threshold T_u, preserving buffer capacity for demand fluctuations. In this implementation, $s = 0.75$ is selected as the operational modeling parameter (reflecting the normal distribution relation $\mathrm{IQR} \approx 2\,\mathrm{MAD}$ relative to a classical 1.5 MAD parameter). When fewer than two utilization samples are recorded, the dynamic policy defaults to the static fallback $T_u = 0.80$. In verified runs, calculated IQR thresholds ranged between 0.72 and 0.73.

5 Methodology and Assumptions

5.1 Terminology and Resource Modeling

- **Processing Element (PE):** An individual virtual or physical processor core.
- **Million Instructions Per Second (MIPS):** The instruction processing throughput of a PE. MIPS denotes computational capacity rather than electrical power.
- **Million Instructions (MI):** The discrete computational work magnitude of a task (*cloudlet*).
- **Requested vs. Allocated MIPS:** Requested MIPS represents workload demand generated by a VM; Allocated MIPS represents the computational service delivered by the host.
- **Host CPU Utilization** $u(t)$: Defined as $\sum$ Allocated MIPS/Host Capacity MIPS $\in [0, 1]$.
- **Active Host:** Any physical host currently powered on and hosting at least one active VM.

5.2 Simulation Topology and Configuration

Simulation Parameter	Configured Value
Simulation Platform	CloudSim Plus 8.0.0 [2], JDK 25
Physical Hosts	4 hosts; 8 PEs × 1000 MIPS (32,000 MIPS total datacenter capacity)
Host Memory / Bandwidth	16,384 MB RAM; 10,000 bandwidth units
Power Model	Linear: $P_{\text{idle}} = 175$ W (static power), $P_{\max} = 250$ W (peak power)
Virtual Machines	8 VMs; 2 PEs × 1000 MIPS; 2048 MB RAM; 1000 bandwidth units
Cloudlets (Workloads)	8 cloudlets; 20,000,000 MI per PE (40,000,000 MI total work per VM)
Workload Profile	Synthetic 24-hour diurnal traces formatted to PlanetLab specifications
Scheduling Interval	300 seconds (5 minutes, matching trace sampling period)
Simulation Termination Cap	86,400 seconds (24 hours)
VM Selection Policy	Minimum Migration Time (MMT)

Table 1: *Architectural and simulation parameters common to both baseline experiments.*

Workload Formulation. Each VM is assigned an independent trace consisting of 288 data points (sampled every 5 minutes across 24 hours). For this verification phase, synthetic diurnal curves exhibiting peak-day and valley-night characteristics were utilized. In subsequent phases, full real-world PlanetLab traces will be deployed.

Implementation Refinements. During initial baseline validation, CloudSim Plus's default resource provisioner caused memory and bandwidth tracking collisions when candidate destination hosts temporarily evaluated migrating VM objects. An independent provisioner (`HostLocalResourceProvisioner`) was introduced to manage allocations strictly per-host. Additionally, CPU utilization was calibrated against total physical capacity rather than requested MIPS, preventing unphysical negative thresholds.

5.3 Performance Metrics

The evaluation framework adheres to the cloud resource management metrics defined by Beloglazov and Buyya [1]:

Energy Consumption E (J, kWh). Total electrical energy integrated across all active hosts:

$$E = \sum_{h} \sum_{k:\, h \text{ active}} P(\bar{u}_{h,k})\, \Delta t_k, \qquad 1 \text{ kWh} = 3.6 \times 10^6 \text{ J}, \tag{2}$$

where $P(u) = 175 + 75 \cdot u$, and $\bar{u}_{h,k}$ represents the average utilization during interval Δt_k. The energy calculation spans the recorded active sample window (approximately 300.1 s to cloudlet completion at $\sim$ 46,500 s).

SLATAH (SLA Time per Active Host). Fraction of active host execution time during which physical CPU capacity is fully saturated:

$$\text{SLATAH} = \frac{1}{N} \sum_{i=1}^{N} \frac{T_{s_i}}{T_{a_i}}, \tag{3}$$

where N is the active host count, T_{a_i} is active host duration, and T_{s_i} denotes the duration where host utilization $u \geq 0.999$. The original definition uses 100% utilization [1]; 0.999 tolerates floating-point rounding. Saturation means no remaining CPU headroom; on its own it does not prove that demand was unmet. Policy thresholds (0.80 for THR, about 0.72 for IQR) trigger migrations below saturation, so SLATAH can be zero even when migrations occur.

PDM (Performance Degradation due to Migrations). Workload execution degradation induced by live migration memory copying:

$$\text{PDM} = \frac{1}{M} \sum_{j=1}^{M} \frac{C_{d_j}}{C_{r_j}}, \qquad C_{d_j} = 0.10 \cdot \text{MIPS}_j \cdot t_{m_j}, \tag{4}$$

where $M = 8$ VMs, $\text{MIPS}_j = 2000$ MIPS (2 PEs $\times$ 1000 MIPS), t_{m_j} is total migration duration in seconds, and $C_{r_j} = 40{,}000{,}000$ MI is the completed requested cloudlet work across all PEs. Using completed work is the project's approximation of the requested-capacity denominator in [1].

SLAV (Overall SLA Violation). Composite metric capturing both saturation and migration overhead:

$$\text{SLAV} = \text{SLATAH} \times \text{PDM}. \tag{5}$$

ESV (Energy and SLA Violations). Unified metric balancing energy conservation against SLA preservation:

$$\text{ESV} = E_{\text{kWh}} \times \text{SLAV}. \tag{6}$$

6 Preliminary Results

Table 2 summarizes the comparative performance of the verified baseline runs. Figures 1 and 2 present the corresponding terminal output captures.

Metric	Static Threshold (THR)	Dynamic Statistical (IQR)
Energy Consumption (Joules)	35,311,961.05 J	35,311,761.57 J
Energy Consumption (kWh)	9.8089 kWh	9.8088 kWh
SLATAH (SLA Time per Active Host)	0.000000	0.000000
PDM (Degradation due to Migration)	4.096×10^{-6}	4.096×10^{-6}
SLAV (Overall SLA Violation)	0.000000	0.000000
ESV (Energy $\times$ SLAV)	0.000000	0.000000
Completed VM Migrations	2	2
VMs Created / Expected	8 / 8	8 / 8
Cloudlets Completed / Expected	8 / 8	8 / 8

Table 2: *Single-run comparative benchmark results under synthetic diurnal workload traces.*

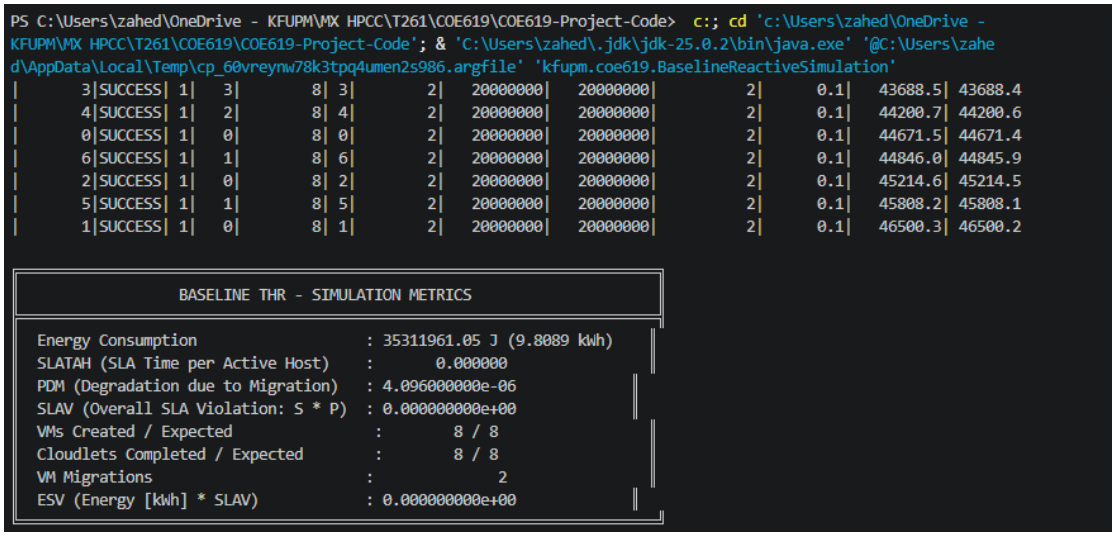


Figure 1: *Terminal simulation results and metrics for Static Threshold (THR baseline).*

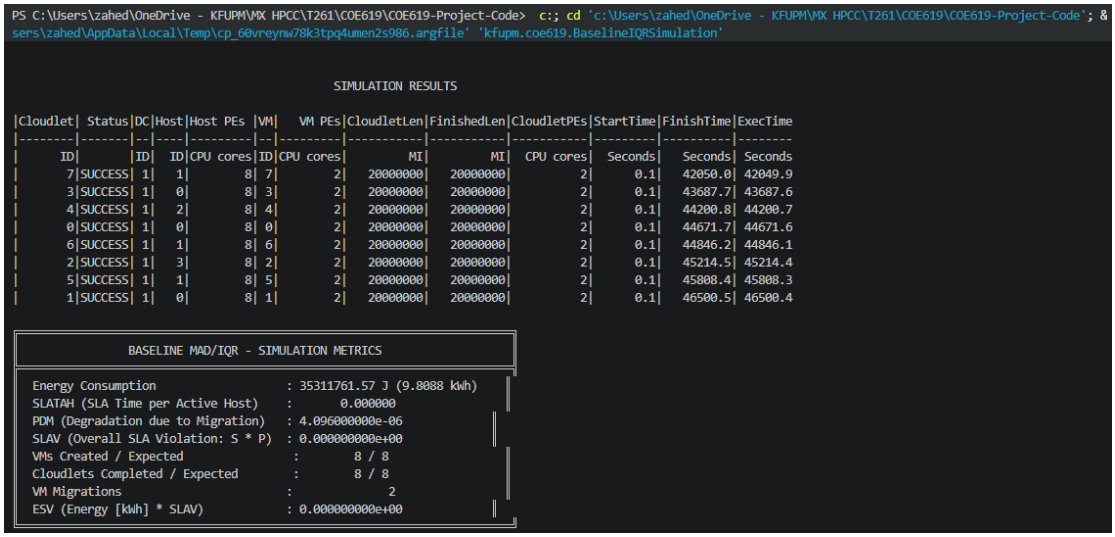


Figure 2: *Terminal simulation results and metrics for Interquartile Range (IQR baseline).*

Discussion. Both baseline simulations completed successfully with 100% task and VM accounting. Each policy performed two VM live migrations. Energy differs by about 199.48 J (0.000565%); the cause of this small difference has not been established, and repeated runs of equivalent code have also shown last-digit variation. No host reached saturation, so SLATAH, SLAV, and ESV are zero; these are computed values, not forced. PDM is positive because two migrations occurred, under the assumed 10% penalty. A single run per policy does not show that either policy is better.

7 Limitations

- **Workload Scale:** Evaluations utilized synthetic diurnal curves across 4 hosts; statistical generalization requires large-scale real-world PlanetLab traces.
- **Energy Integration Window:** Reported energy reflects the active execution window ($\sim$ 12.92 hours) rather than an extrapolated 24-hour idle baseline.
- **Single-Run Baseline:** Reported figures represent single-run operational verification; multi-

run statistical distributions are necessary for definitive conclusions.

- **Metric Approximations:** SLATAH uses $u \geq 0.999$ as the saturation condition. PDM relies on an assumed 10% migration penalty and uses completed cloudlet work as its denominator. SLAV and ESV are composite research metrics [1], not contractual SLA measures.

8 Progress Against Plan and Next Steps

Task Description	Proposal Plan	Status	Milestone
Literature review & simulation setup	Term 261, Week 6	Completed	CloudSim Plus verified
Baseline implementation & validation	Term 261, Week 9	Completed	THR and IQR operational
Progress Report I	Term 261, Week 10	This report	Baseline results
Predictive model mathematical design	Term 261, Week 13	Next	Algorithm formulation
Progress Report II	Term 261, Week 15	Planned	Algorithm design report
Predictive algorithm implementation	Term 262, Week 4	Planned	CloudSim Plus module
Final Prototype and Demonstration	Term 262, Week 12	Planned	Live demonstration

Table 3: *Project execution progress evaluated against the proposal schedule.*

Immediate Next Steps:

1. Conduct repeated multi-run experiments to quantify variation and evaluate saturation under varied workload densities.
2. Scale up simulation topology and integrate official PlanetLab CPU traces [6].
3. Design the predictive overload forecasting model (e.g., Simple Linear Regression or Exponential Smoothing) for Progress Report II.

References

[1] A. Beloglazov and R. Buyya, "Optimal online deterministic algorithms and adaptive heuristics for energy and performance efficient dynamic consolidation of virtual machines in cloud data centers," *Concurrency and Computation: Practice and Experience*, vol. 24, no. 13, pp. 1397–1420, 2012, doi:10.1002/cpe.1867.

[2] M. C. Silva Filho, R. L. Oliveira, C. C. Monteiro, P. R. M. Inácio, and M. M. Freire, "CloudSim Plus: A cloud computing simulation framework pursuing software engineering principles for improved modularity, extensibility and correctness," in *Proc. 2017 IFIP/IEEE Symposium on Integrated Network and Service Management (IM)*, 2017, pp. 400–406, doi:10.23919/INM.2017.7987304.

[3] R. N. Calheiros, R. Ranjan, A. Beloglazov, C. A. F. De Rose, and R. Buyya, "CloudSim: A toolkit for modeling and simulation of cloud computing environments and evaluation of resource provisioning algorithms," *Software: Practice and Experience*, vol. 41, no. 1, pp. 23–50, 2011, doi:10.1002/spe.995.

[4] C. Clark, K. Fraser, S. Hand, J. G. Hansen, E. Jul, C. Limpach, I. Pratt, and A. Warfield, "Live migration of virtual machines," in *Proc. 2nd USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2005, pp. 273–286.

[5] W. Voorsluys, J. Broberg, S. Venugopal, and R. Buyya, "Cost of virtual machine live migration in clouds: A performance evaluation," in *Proc. 1st International Conference on Cloud Computing (CloudCom)*, LNCS 5931, Springer, 2009, pp. 254–265, doi:10.1007/978-3-642-10665-1_23.

[6] K. Park and V. S. Pai, "CoMon: A mostly-scalable monitoring system for PlanetLab," *ACM SIGOPS Operating Systems Review*, vol. 40, no. 1, pp. 65–74, 2006, doi:10.1145/1113361.1113374.

[7] D. C. Marinescu, *Cloud Computing: Theory and Practice*, 3rd ed. Morgan Kaufmann, 2022.